

CSG102: UNIX & SHELL PROGRAMMING

ULTIMATE EXAM CRASH-COURSE NOTES

Exam Blueprint Awareness: Total Marks: 100. Internal Evaluation (40%) consists of Attendance (10%), Homework/Assignments (10%), and Quizzes/Presentations (20%). Mid-Term Examination (MTE) accounts for **20%** (Best of 2 MTEs), and the End-Term Examination (ETE) accounts for **40%**. This guide maps perfectly to your official syllabus units A through F.

Unit A: Introduction to Linux & Architecture

An **Operating System (OS)** is a core piece of software that acts as an intermediary bridge between the physical computer hardware and the user/applications. It manages hardware resources (CPU, Memory, Disks) and provides an environment to execute programs seamlessly.

1. Core Architecture of Unix/Linux

The architecture consists of four distinct layered rings working in harmony:

- **Hardware:** The physical layer comprising the CPU, RAM, hard disk drives, and peripheral devices.
- **Kernel:** The absolute heart/core of the operating system. It interacts directly with the hardware. It handles memory management, process scheduling, file management, and device control. Regular applications cannot touch hardware directly; they ask the kernel to do it via **System Calls**.
- **Shell:** The command interpreter. It acts as an interface between the user and the Kernel. It accepts commands typed by the user, interprets them, and passes them to the kernel for execution. Examples include Bash, Sh, Csh, and Ksh.
- **Utilities/Applications:** Standard utility programs and user applications (like compilers, editors, web browsers, or basic commands like `ls`, `grep`) that run on top of the shell.

2. Key Features of Unix/Linux

- **Multi-user System:** Multiple users can access the system's resources simultaneously without interfering with each other.
- **Multi-tasking:** Capable of executing multiple distinct processes or tasks concurrently by dividing CPU time efficiently.
- **Portability:** Written primarily in the high-level C programming language, making it easy to compile and run across wildly diverse hardware platforms.
- **Hierarchical File Structure:** Files and directories are organized logically in an inverted tree structure starting from the root directory (`/`).

- **Security & Permissions:** Implements robust ownership models, granting read, write, and execute permissions independently to the owner, group, and others.

3. General Purpose Essential Commands

Command Syntax	Description & Functional Utility	Practical Example Output Structure
<code>date</code>	Displays the current system date, time, and timezone setting.	<code>Wed May 27 17:55:12 IST 2026</code>
<code>cal</code>	Renders a text-formatted calendar visual grid for the current month.	Prints a structural grid calendar of May 2026
<code>who</code> / <code>whoami</code>	Lists users currently active on the host / displays current logged-in identity.	<code>yavour tty1 May 27 08:30</code>
<code>echo [text]</code>	Prints string arguments out to standard output (screen).	<code>echo "Hello"</code> outputs <code>Hello</code>
<code>clear</code>	Clears the terminal display screen completely, refreshing the prompt.	Clears visual terminal interface lines

Exam Tip for Unit A: Expect a question on "Explain Unix Architecture with a neat diagram." Draw concentric circles with Hardware at the center, followed by Kernel, Shell, and User Applications on the outermost layer.

Unit B: File Oriented Commands & Regular Expressions

In Unix, handling files and scanning files for specific patterns using filters is a critical skill. Files are manipulated using structural commands, and refined via pattern matching logic.

1. File and Directory Operations

- `ls` : Lists directory contents. Common flags include `-l` (long details showing size, date, permissions) and `-a` (shows hidden files starting with a dot).
- `cat` : Concatenates and displays content of text files. Can also create files using redirection (`cat > newfile.txt`).
- `cp [source] [dest]` : Copies files or directories from one path to another destination.
- `mv [old] [new]` : Moves or renames files or directories.
- `rm` : Removes (deletes) files. Use `-r` carefully to recursively delete entire folders.
- `mkdir` / `rmdir` : Creates a new blank directory / deletes an empty directory.

2. Essential Filters & Utilities

Filters accept input data, process it according to parameters, and output transformed results.

- **more** / **less** : Page-by-page viewing utilities for massive text files, preventing terminal text overflow.
- **wc [file]** : Word count utility. Displays count lines (**-l**), words (**-w**), and bytes/characters (**-c**).
- **diff [file1] [file2]** : Compares two files line by line and outputs exact textual discrepancies.
- **sort [file]** : Rearranges lines of text files alphabetically or numerically (**-n**).
- **uniq** : Drops adjacent duplicate lines from a sorted file. It requires data to be sorted first!

3. Powerful Stream Pattern Filters: grep and sed

grep (Global Regular Expression Print): Scans files for text lines matching a specific pattern.

```
Syntax: grep [options] "pattern" filename
Example: grep -i "unix" syllabus.txt      # -i makes search case-insensitive
Example: grep -n "error" log.txt          # -n shows line numbers containing "error"
```

sed (Stream Editor): A non-interactive stream tool used to search, find-and-replace, insert, or delete specific segments of a text stream dynamically.

```
Syntax: sed 's/old_word/new_word/g' filename
# 's' stands for substitute, and 'g' stands for global replacement across the whole line.
```

4. Regular Expressions (Regex) Meta-characters

Regex defines string search patterns using specialized symbols:

- **^** (Caret): Matches structural line starts. (e.g., **^Linux** matches any line starting with the word "Linux").
- **\$** (Dollar): Matches line ends. (e.g., **done\$** matches any line ending with "done").
- **.** (Dot): Matches any single individual character except newline.
- ***** (Asterisk): Matches zero or more occurrences of the preceding character expression.
- **[abc]** : Bracket expression matching any single character from the internal list (a, b, or c).

Unit C: Linux/Unix File System, Inodes & Mounting

A fundamental Unix philosophy is: **"Everything is a file."** If something isn't a file, it's a process. This includes hard drives, keyboards, printers, and directories.

1. Types of Files in Unix

1. **Regular Files (-)**: Standard text files, binary source files, executable software, images, data documents.
2. **Directory Files (d)**: Special files acting as storage folders containing lists of filenames and their associated pointers (Inodes).
3. **Device/Special Files (c / b)**: Represents physical hardware items under `/dev`. Character devices (c) transfer data character-by-character (e.g., keyboards), while Block devices (b) stream data in large block arrays (e.g., hard disks).

2. Understanding the Inode Structure

An **Inode (Index Node)** is a data structure on the disk that stores all metadata properties about a file. Crucially, *the inode does not store the actual filename or the actual raw file data content.*

An inode contains the following attributes:

- File type (regular, directory, device link).
- File size in bytes.
- Owner ID (UID) and Group ID (GID).
- Access Permissions (Read, Write, Execute).
- Timestamps (Creation, modification, and last access dates).
- Hard Link count counter value.
- **Pointers**: Direct and indirect block addresses on disk where the real data text blocks are located.

3. File System Storage Components

A typical Unix file system partition disk block layout is structured into 4 foundational sectors:

- **Boot Block**: Occupies the initial starting blocks of the disk; contains primary bootstrap loader sequence code to turn on the OS.
- **Super Block**: Stores total health/status information about the filesystem, including total disk size, free block count, total inodes, and free block location trackers.
- **Inode List**: A structured contiguous table array containing all allocated and unallocated Inodes for the block system.
- **Data Blocks**: The actual storage blocks where the real contents of files and directories are physically saved.

4. Mounting and Un-mounting Systems

Mounting is the operational process of attaching an external filesystem partition (like a USB drive or secondary hard disk) to a specific target location directory in the main file system tree structure so it becomes accessible to users.

- **mount /dev/sdb1 /mnt/usb** : Connects hardware block device **sdb1** to directory folder path **/mnt/usb** .
- **umount /mnt/usb** : Safely detaches/disconnects the partition before physically pulling out the drive.

Unit D: Processes in Linux, Scheduling & Signal Handling

A **Process** is simply a program in a state of active execution. When you run a command like **ls** , a temporary process is initialized, performs work, and terminates.

1. Process Creation Mechanism: fork() and exec()

In Unix, new processes are spun up via a strict parent-child relationship framework using system calls:

- **fork() System Call**: Creates an identical child duplicate clone of the calling parent process. The child process gets a brand new unique Process ID (PID) and copies the parent's memory states.
- **exec() System Call**: Overwrites the newly cloned process's memory space with a completely new executable binary program code (e.g., loading a different utility program).

2. Initialization Sequences: rc and init

- **init Process**: The absolute grandparent ancestor of all active processes on a Unix machine. It carries a designated constant Process ID **PID = 1**. It initializes the system environment components immediately following the kernel boot up sequence.
- **rc Files (Run Control)**: Initialization automation scripts located inside **/etc/rc.d** or similar directories that dictate exactly what background system processes and network services start up automatically depending on systemic run levels.

3. Job Control Management

Users run processes in two operational contexts within a interactive shell prompt:

- **Foreground Processes**: Take direct control of the terminal window; the user must wait for completion before submitting a new command.
- **Background Processes**: Runs silently behind the active interface, leaving the prompt immediately free for secondary inputs. To launch a task directly to the background, simply append an ampersand (**&**) symbol to the command end.

```
Example: long_backup_script.sh &
# This script runs in the background. Check running tasks using 'jobs' command.
```

Key automation tools include:

- **cron** : A system background daemon used to schedule recurring automation scripts at specific times, hours, or calendar dates based on configuration rules inside a file called **crontab** .
- **at** / **batch** : Schedules a one-time execution of a targeted script command at a specific future time offset.

4. Process Signals and Termination Handling

Signals are asynchronous operational interrupts sent to a running process to force automated behavioral reactions (like pausing or quitting immediately).

- **kill -9 [PID]** : Sends the absolute unblockable **SIGKILL** signal, terminating the process immediately.
- **Ctrl + C** : Sends a **SIGINT** (Interrupt) signal from the keyboard to cleanly kill a foreground program.
- **Ctrl + Z** : Sends a **SIGTSTP** (Terminal Stop) signal, suspending the running process into the background until revived via **fg** or **bg** commands.

Unit E: Shell Programming (Scripting Essentials)

A **Shell Script** is nothing but a plain text file containing a structured sequential list of shell commands executed in order. It incorporates standard programming conventions like variables, loops, and logical condition tests.

1. Core Syntax Foundations

- **Shebang line (#!/bin/bash)** : The very first entry line inside any script. It instructs the Unix kernel exactly which shell interpreter environment to allocate to parse this script.
- **Variables** : Declared dynamically without any explicit type assignments. Do not include spaces around the assignment equals sign. Reference variable values using a dollar sign (**\$**).

```
name="Yavour"
echo $name
```

2. Scripting Control Flow Structures

Conditional Statements (If-Else Checking): Note the mandatory spaces inside the brackets.

```
if [ $num -gt 10 ]; then
    echo "Number is greater than 10"
```

```
else
    echo "Number is smaller or equal"
fi
```

Operators for comparison: **-eq** (Equal to), **-ne** (Not equal), **-gt** (Greater than), **-lt** (Less than).

Looping Iterators (For and While Blocks): Used to repeat actions.

```
# Example While Loop counting down
while [ $count -gt 0 ]; do
    echo "Count is $count"
    count=`expr $count - 1`
done
```

Unit F: The C Environment, Descriptors & Advanced Concepts

Unix has deep roots intertwined natively with the C language development environment. This unit reviews how C interfaces with low-level file concepts and basic descriptor metrics.

1. File Descriptors and Standard Streams

When a process launches, the kernel opens three basic default systemic communication channels/streams for it automatically, tracking them via integer markers called **File Descriptors (FD)**:

- **Standard Input (stdin):** File Descriptor **0**. Handles keyboard character entries.
- **Standard Output (stdout):** File Descriptor **1**. Handles normal application print out text stream text on the terminal window.
- **Standard Error (stderr):** File Descriptor **2**. Independent error text diagnostic stream, printing error alerts separately from standard outputs.

2. Advanced Descriptor Parameters

Syllabus sections require high-level familiarity with structural descriptors used in system design:

- **Simple Descriptors:** References raw operational components tracking data fields.
- **Fourier / Regional / Texture Descriptors:** Advanced algorithmic shape and pattern boundary representations used primarily within analytical Unix computer vision subsystems or image segmentation matrices (Edge detection, thresholding boundaries).

Mandatory Exam Laboratory Script Solutions

Here are clear, highly-commented shell script and C program code structures matching your exact laboratory mandatory exam topics list.

Lab 1: Shell script to check if a number is Even or Odd

```
#!/bin/bash
# Lab Requirement 5: Check whether a given number is even or odd
echo -n "Enter an integer number: "
read num

# Use modulo operation to find remainder when divided by 2
remainder=$(( num % 2 ))

if [ $remainder -eq 0 ]; then
    echo "The entered number $num is EVEN."
else
    echo "The entered number $num is ODD."
fi
```

Lab 2: Shell script to count the number of files in a directory

```
#!/bin/bash
# Lab Requirement 9: Count number of files inside a Directory
echo -n "Enter targeted directory path: "
read dir_path

if [ -d "$dir_path" ]; then
    # ls lists files, wc -l counts total lines/items
    file_count=$(ls -l "$dir_path" | grep -v "^d" | wc -l)
    echo "Total number of files in $dir_path is: $file_count"
else
    echo "Provided path is not a valid directory!"
fi
```

Lab 3: Shell script to display "HELLO WORLD" and track active users

```
#!/bin/bash
# Lab Requirement 3 & 2: Print Hello World & display currently logged-in users
echo "HELLO WORLD"
echo "-----"
echo "The currently active logged-in users are:"
who
```

Lab 4: C Program to emulate the Unix "ls -l" long listing command attributes

```
#include <stdio.h>
#include <sys/stat.h>
#include <sys/types.h>
#include <unistd.h>

// Lab Requirement 7: Emulate Unix ls -l command metrics for a target file
int main() {
    struct stat file_status;
    char path[] = "testfile.txt";

    // Obtain file metrics using stat system call
    if (stat(path, &file_status) == 0) {
        printf("File Statistics for: %s", path);
        printf("-----");
        printf("File Size: %ld Bytes", file_status.st_size);
        printf("Number of Hard Links: %ld", file_status.st_nlink);
        printf("Owner User ID (UID): %d", file_status.st_uid);
        printf("Group Identifier ID: %d", file_status.st_gid);
        printf("Inode Address Number: %ld", file_status.st_ino);
    } else {
        printf("Error: Could not fetch stats for file %s", path);
    }
    return 0;
}
```